以物料平衡为原则的生产过程 0-1 决策

摘　要

本文针对生产过程中的决策问题，在仔细研究了工程背景和应用实际之后，建立了关于生产过程决策变量最优求解的数学模型，并在多种实际工况下验证了模型的合理性，并对模型进行了敏感性分析，讨论了模型的鲁棒性，我们的模型在实际生产中具有一定的指导意义。

针对问题一的情形 1，我们首先假设样本中的次品数 X 满足**二项分布**，通过计算抽样数为 n=i 时检测出的样品中大于等于 j 个为次品的概率 $P_i(X \geqslant j)$ 列出了关于抽样数量和统计量临界值的表格，然后我们构造了**显著水平** $\alpha = 0.05$ 的**单侧假设检验**，通过找到显著水平最接近 5% 的 i，j 值，找到其中 i 最小的一组解就是我们设计的抽样检测方案。具体方案为：随机抽取 16 个零件作为样本，对它们进行检测，若出现 2 个及以上的次品，则拒收这批产品。详细结果见表 1。针对问题 1 的**情形 2**，我们构造了 $\alpha = 0.1$ 的单侧假设检验，我们通过计算 $P_i(X \leqslant j)$ 也得到了关于抽样数量和统计量临界值的表格。具体方案为：随机抽取 20 个零件作为样本，对它们进行检测，若未出现任何次品，则接收这批产品。详细结果见表 2。**针对问题二**，我们首先构建了一系列决策变量，并基于**物料平衡原则**建立数学模型。通过对流程图的分析，我们得到了三类约束条件，分别为总流量约束、次品流量约束以及决策变量的自我约束。接着，我们设定目标函数为最小化生产一件合格品的成本，得到 **0-1 规划**问题。为了求解这一优化问题，我们使用了**枚举法**和遗传算法。在六种不同情况下，得到的最低成本分别为：37.6917 元/件、46.3275 元/件、39.7216 元/件、43.0298 元/件、43.2618 元/件和 34.2980 元/件，单位为每出售一件合格成品。结果记录在 *problem2.xlsx* 中。**针对问题三**，与问题二类似，但多了半成品环节。我们同样首先构建决策变量，并根据物料平衡原则建立模型，对流程进行分析也得到了三类约束条件，约束条件依然是总流量约束、次品流量约束以及决策变量自我约束。目标函数依然是最小化生产一件合格品的成本。然而，由于本问题更复杂，第二问中使用的穷举法不再适用，因此我们采用**遗传算法**进行求解。在题目给定条件下，计算得到的最低成本为 143.0543 元/件，单位为每出售一件合格成品，结果记录在 *problem3.xlsx* 中。

针对问题四，我们按照问题一情形 1 的置信度计算了次品率分别为 0.1 和 0.2 时的**置信区间**，并依据置信区间的上下界对模型进行**敏感性分析**，具体讨论了对次品率波动敏感和不敏感的决策变量。并且对次品率实际值偏高和偏低的情况对模型决策产生的影响进行了理论分析，通过实验验证了我们的理论猜想。结果见 *problem4_case.xlsx*。

关键词：0-1 整数规划　非线性规划　物料平衡原则　遗传算法　生产过程决策　敏感性分析

一、问题重述

1.1 问题背景

生产装配过程中，不同的决策可能对成本产生不同显著程度的影响，仅依靠经验进行决策会带来隐形的多余成本损失。有效优化企业在零部件采购、产品组装及不合格品的处理方法一系列过程中所产生的成本尤为重要。

在装配的成品中，只要其中一个零配件不合格，则成品一定不合格；如果两个零配件均合格，装配出的成品也不一定合格。基于此条件，我们需要对是否检测零件、成品，是否对不合格成品进行拆解等决策进行建模，通过定量优化分析得出最优决策。

1.2 问题的提出

1.2.1 问题一的提出

企业要购买一批零件，现准备检查要购买的一批零配件次品率是否会超过某标称值。给定标称值为 10%，分别在以下两种情况设计出一个抽样检测方案, 使得检测次数最少：

（1）如果在 95% 的置信度下零配件的次品率超出了标称值，则拒绝接收这批零配件。

（2）如果在 90% 的置信度水平下零配件的次品率没有超过标称值，则接收这批零配件。

1.2.2 问题二的提出

企业生产过程大致分为四个部分，

（1）是否对零配件（1 或 2）进行检测：是，则将不合格的丢弃；否，则直接装配。

（2）是否对成品进行检测：是，则只有检测合格的成品进行销售；否，则直接销售。

（3）对从（2）检测得到的不合格成品，则分为是否拆解两部分，若拆解，则对拆解的零件重复（1）,（2）；若不拆解，直接丢弃。

（4）顾客如购买到不合格成品，则企业一定调换，并有一定的调换费用。对调换所得的不合格成品，重复（3）。

现给出六种不同的生产情况，每种情况分别给出了零件与成品的次品率、购买单价、检测成本，以及不合格成品的调换损失与拆解费用，对每种情况给出合理的决策方案使得企业平均利润最大。

1.2.3 问题三的提出

给定一个产品所需 8 个零配件，2 道工序，现给出一种具体的生产情况（包括次品率，检测成本等值），同样，重复问题二，给出合理的决策方案使得企业平均利润最大。

1.2.4 问题四的提出

若问题二、三的次品率不是真值，是由抽样检测方法所得到的，需要重新考虑问题二、问题三中的决策选择，尽量避免抽样检测所带来的误差。

二、问题的分析

2.1 问题一的分析

为了设计抽样检测的方案，并且达到要求的信度，我们可以采用假设检验的方法，已知标称值 $p = 10\%$，我们假设样品中次品数为满足 $X \sim b(n, p)$ 的二项分布。在样本量较大的情况下，二项分布可以使用正态分布进行近似。一般来说，需要满足 $npq > 20$ 才可使用此近似。在此问题中，$p = 0.1, q = 0.9$，这意味着样本量至少要达到 223，而这显然已经超出了预期范围，不符合“检测次数尽可能少”的要求。因此，本问题不采用正态分布近似，而是计算精确概率值。

对于情形 1：在 95% 的信度下认定零配件次品率超过标称值，则拒收这批零配件。我们构造显著水平为 $\alpha = 0.05$ 的假设检验问题，原假设 H_0 为次品率不超过标称值，备择假设 H_1 为次品率超过标称值。设拒绝域为 $X \geqslant j$，通过计算抽样数 $n = i$ 时检测出的样品中大于等于 j 个为次品的概率 $P_i(X \geqslant j)$，我们可以列出一张有关抽样数量与统计量临界值的表格，表格中的值 $P_i(X \geqslant j)$ 即为拒绝域为 $X \geqslant j$ 的假设检验的显著水平。在样本量 i 尽量小的前提下，显著水平最贴合 5% 且不超过 5% 的 i, j 值代表了我们想得到的抽样检测方案，即抽取 $n = i$ 个样本，若其中有大于 j 个样本为次品，则拒绝原假设，也就是拒收此批零配件。

对于情形 2：在 90% 的信度下认定零配件次品率不超过标称值，则接收这批零配件。同理，构造显著水平为 $\alpha = 0.1$ 的假设检验问题，原假设 H_0 为次品率超过标称值，备择假设 H_1 为次品率不超过标称值。设拒绝域为 $X \leqslant j$，当且仅当拒绝原假设时，接受备择假设，也即接收这批零配件。同样列表计算 $P_i(X \leqslant j)$ 并寻找贴近 10% 且不超过 10% 的值，可以确定抽样检测的方案。

2.2 问题二的分析

对于第二问，我们可以基于物料平衡的原则，首先，我们需要构建一系列不同的 0-1 决策变量，主要有零配件、成品是否检测以及是否拆解三类变量，设置好一系列变

量后我们可以基于物料平衡的原则来建立模型。具体来讲，总共的约束分为三种约束，分别是总流量约束，次品流量的约束以及决策变量自身的约束，我们需要通过对流程图进行进一步分析来得到一系列约束等式，其中等式的个数要与变量个数相同。

为了公平的比较每种决策方案的优劣，我们令目标函数是售出一件合格品过程所产生的所有成本，我们需要最小化这个成本。最后我们需要将目标函数和约束整理为一个规划问题，由于决策变量与非决策变量之间满足一一对应的关系，即一组决策变量值能确定流程图中的次品率以及流量，我们固定决策变量，显式求解此刻的非决策变量约束方程，这一过程可以通过最小化方程表达式的均方误差实现，我们得到了该情况下的流程图运行情况，并据此计算目标函数，在优化过程中最小化此目标函数以得到最优决策。

2.3 问题三的分析

对于第三问，类似第二问，我们可以先构建一系列决策变量，其中需要包含半成品的检验和拆解决策，之后同样按照物料平衡的原则来建立模型。同样也有三种约束：总流量约束，次品流量的约束以及决策变量自身的约束，对流程进行分析后我们可以得到全部约束，同样，我们目标仍然是最小化一件合格成品的生产成本。对于求解最优化问题，通过使用第二问的通过穷举法寻找最优决策变量组合的方法不再适用，我们可以使用一些启发式搜索方法比如遗传算法来进行求解。

2.4 问题四的分析

问题四的本质是优化问题的敏感性分析。当次品率是由抽样检测得出的时，它并不能直接代表真实的次品率。要想获得真实次品率的大致范围，我们可以构造置信区间。置信区间可以代表较有把握的实际次品率范围。然后，我们需要分析当真实的次品率处于这个范围之内的情况下，生产模型中的决策变量的最优组合相比原模型是否收到影响，以及受到怎样的影响。具体来说，我们应当分别分析每一个次品率参数发生变化时，模型的解的变化。当研究某个次品率参数的影响时，我们将它分别设置为置信上界和置信下界，其他次品率仍然保持原次品率（如 10%）不变，重新求解模型。

三、模型的假设

1. 假设装配和拆解过程不会对零件或半成品造成损坏
2. 假设问题 2、3 中的次品率为真值
3. 假设进入市场的成品会立刻被卖出
4. 假设对零件、半成品、成品的检测总是准确的

四、符号说明

将模型中的所有符号列出如下。由于问题二和三部分变量含义不同，因此将两个问题的符号分列两表。每张表分为上下两个部分，其中上半部分包含 0-1 决策变量（粗体）以及待求解的连续变量，下半部分为已知常量。

4.1 问题二符号说明

见表 1

符号	说明
$\boldsymbol{t_i}$	是否对输入产线的零件 i 进行检验
$\boldsymbol{t'_i}$	是否对由成品拆解出的零件 i 进行检验
$\boldsymbol{t_p}$	是否对成品进行检验
$\boldsymbol{d_p}$	检验不合格的成品是否拆解
n_i	零件 i 的采购量
n_p	成品的生产量
l_i	由成品拆解并回流产线的零件量
$\hat{e}_i$	用于生产成品的零件的次品率
e'_i	由成品拆解返回零件的次品率
$\hat{e}_p$	生产出的成品的实际次品率
p_i	零配件 i 的购买单价
tc_i	零配件 i 的检测成本
tc_p	成品的检测成本
ass	装配成本
$dass$	拆解费用
re	调换损失
e_i	输入零件的次品率
e_p	零件完好的情况下组装成品的次品率

表 1

4.2 问题三符号说明

见表 2

符号	说明
$\boldsymbol{t_i}$	是否对输入产线的零件 i 进行检验
$\boldsymbol{t'_i}$	是否对从生产出的次品半成品拆解出的零配件 i 进行检验
$\boldsymbol{t''_i}$	是否对从拆解得的半成品拆解出的零配件 i 进行检验
$\boldsymbol{t_{p_j}}$	是否对生产出的半成品 j 进行检验
$\boldsymbol{t'_{p_j}}$	是否对从成品拆解得的半成品 j 进行检验
$\boldsymbol{t_p}$	是否对生产出的成品进行检验
$\boldsymbol{d_{p_j}}$	检验不合格的半成品 j 是否拆解（半成品生产后的检验）
$\boldsymbol{d'_{p_j}}$	检验不合格的半成品 j 是否拆解（成品拆解出的半成品的检验）
$\boldsymbol{d_p}$	检验不合格的成品是否拆解
n_i	零件 i 的采购量
n_{p_j}	半成品 j 的生产量
n_p	成品的生产量
l_i	由生产出的次品半成品拆解并返回半成品加工步骤的零配件 i 的量
l'_i	由拆解得到的次品半成品拆解并返回半成品加工步骤的零配件 i 的量
l_{p_j}	由次品成品拆解并返回成品加工步骤的半成品 j 的量
$\hat{e}_i$	用于生产半成品的零配件 i 的次品率
$\hat{e}_{p_j}$	生产出的半成品 j 的实际次品率
$\hat{e}_p$	生产出的成品的实际次品率
e'_i	由生产出的半成品拆解返回零件的次品率
e''_i	由拆解出的半成品拆解返回零件的次品率
e'_{p_j}	由成品拆解出的半成品 j 的次品率
$\hat{e}^*_{p_j}$	用于生产成品的半成品 j 的次品率
$\hat{e}^*_i$	用于生产成品的半成品中含次品零配件 i 的比率
p_i	零配件 i 的购买单价
tc_i	零配件 i 的检测成本
tc_{p_j}	半成品 j 的检测成本
tc_p	成品的检测成本
ass_{p_j}	半成品 j 的装配成本
ass_p	成品的装配成本

$dass_{p_j}$	半成品 j 的拆解费用
$dass_p$	成品的拆解费用
re	调换损失
e_i	输入零件的次品率
e_{p_j}	零件完好的情况下组装半成品 j 的次品率
e_p	半成品完好的情况下组装成品的次品率

表 2

五、模型的建立与求解

5.1 问题一的模型建立与求解

我们要求：构造的假设检验的显著性水平与题目要求的值相差不得超过 5%。

1、情形 1：显著性水平应在 0.0475 到 0.05 之间。查询我们列出的概率表可得，样本量 $n = 16$ 时，$P_{16}(X \geqslant 2) \approx 0.485$，符合要求。对应的方案为：随机抽取 16 个零件作为样本，对它们进行检测，若出现 2 个及以上的次品，则拒收这批产品。

2、情形 2：显著性水平应在 0.095 到 0.1 之间。查表可得，样本量 $n = 20$ 时，$P_{20}(X \leqslant 0) \approx 0.984$，符合要求，对应的方案为：随机抽取 20 个零件作为样本，对它们进行检测，若未出现任何次品，则接收这批产品。

可以发现，情形二虽然要求的信度更低，但实际对零配件次品率的要求更严苛，这是因为在假设检验中，保证第一类错误概率在很低的值（如此处的 0.05 和 0.10），会使得拒绝域较小，也即不轻易拒绝原假设。于是，在情形 1 中，原假设为次品率 0.1，要求有充分的证据表明次品率高于 0.1，即拒收零配件；而在情形 2 中，原假设为次品率不低于 0.1，要求有充分的证据表明次品率低于 0.1 才拒绝原假设，即接收零配件。问题一求解思路大致如下图所示

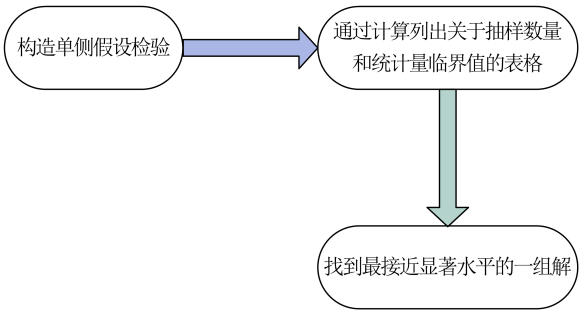

图 1 问题 1 求解思路

5.2 问题二的模型建立与求解

针对问题二和问题三，我们基于物料平衡原则，分别建立了生产模型。物料平衡原则是指在生产过程中，生产出的产品数量应该等于消耗的原材料数量，具体来说，在任何一个节点上，流入的物料数量应当等于流出的物料数量，节点本身不能有物料的累积。对于生产模型的各个节点，都可以从总流量以及次品流量两个方面进行考虑。求解列出的方程组即可获得生产线在稳定状态下的物料消耗水平，以及各个环节的次品率。

问题二、三求解思路大致如下图所示

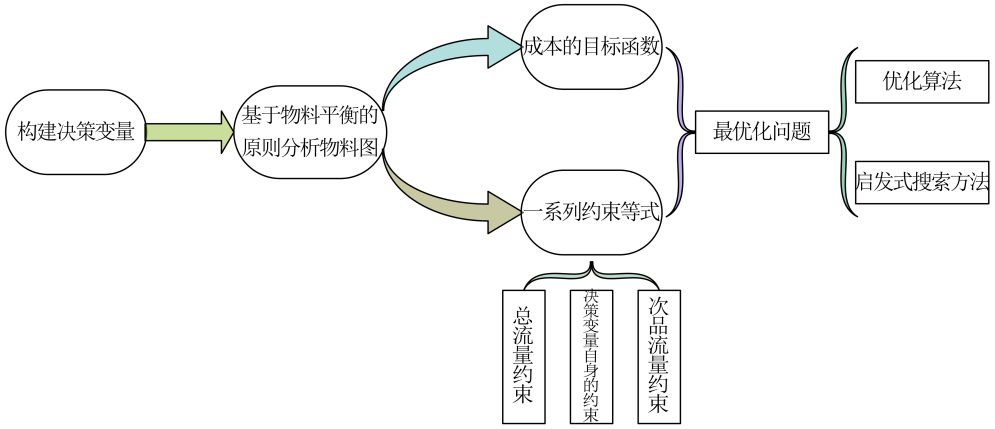

图 2 问题 2，3 求解思路

5.2.1 构造决策变量

1、**零配件的检测**。零配件的检测分为两种情况，分别是输入零配件的检测和由成品拆解出的零配件的检测。这两部分零配件不能进行同等对待，否则将造成资源浪费等问题。例如，零件 1 若在进入系统时经过了检测，则由其生产出的成品拆卸出的零件 1，可以确定是合格品而不需要再次检测。因此，我们考虑将这两种情况区别对待，设置不同的 0-1 决策变量 t_i 和 t_i'，分别表示是否对输入产线的零件 i 进行检验和是否对由成品拆解出的零件 i 进行检验，1 代表进行检测。

2、**成品的检测**。生产出的成品是否检测用 t_p 表示。若不进行检测，则直接进行销售，可以减少检测成本，但可能产生换货造成的额外损失。

3、**是否拆解**。成品的退还有两个途径，一是成品检测的不合格品，二是退换货返回的不合格品。容易看出，这两种来源的不合格品是完全等价的，其是否拆解将对系统效益产生相同的影响。因此，我们只设置一个 0-1 决策变量 d_p，表示是否拆解成品。

由此，我们可以画出问题二的产线的流程如图 3。**流程图如下：**

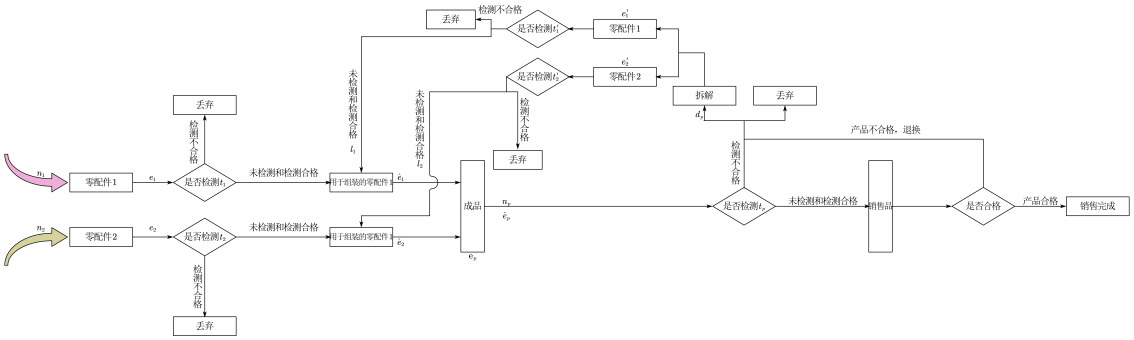

图 3 问题 2 的生产流程图

问题二生产的具体流程大致如下，我们以零配件 1 为例，其数量为 n_1，次品率为 e_1，首先判断是否检测（t_1），若 t_1 为 1，则检测。反之，不检测。之后未检测和经过检测

合格的零件将和从成品拆解所得到的未检测和检测合格的零件来用来组装成成品。组装成成品之后我们需要判断是否检测成品（t_p）, 若检测不合格，这里我们将不合格的成品记为 A；若未检测或检测合格，则变成销售品来卖给顾客。如果是合格的产品，则销售成功；如果是不合格的产品，则会有调换环节 (有额外的调换费用)，这里的不合格的产品将会与 A 一起来进行判断是否拆解 (d_p)，若不拆解，则丢弃；若拆解，则会得到零配件，之后我们会对拆解所得到的零配件 1 再次进行判断是否检测（t_1'），若检测不合格，则丢弃，若未检测或检测合格，则将作为一部分用于组装的零配件。

5.2.2 根据流量构造约束方程

按照变量说明表的方式设置变量之后，我们根据物料平衡原理列出方程。

根据分析，我们从总流量和次品流量两个角度寻找产线的流量约束。**首先考虑总流量约束**。用于加工成品的零件数等于生产出的成品数，即：

$$n_i \cdot (1 - e_i \cdot t_i) + l_i = n_p \tag{1}$$

拆解得到的零件中与实际回流产线的零件数关系：

$$np \cdot dp \cdot \hat{e_p} \cdot (1 - e_i' \cdot t_i') = l_i \tag{2}$$

生产出的产品，不论在销售前是否进行检测，最终仅有合格品能够销售成功。为了便于计算销售成功一件成品所用的成本，我们此处规定销售成功量为 1：

$$np \cdot (1 - \hat{e_p}) = 1 \tag{3}$$

观察上述方程，用于描述总体流量的变量包含 n_1, n_2, l_1, l_2, n_p 共 5 个，方程数量为 5 个。

接下来考虑次品流量的约束。

流入和流出产线的各零配件次品量相同。输入产线的零配件中的次品将和拆解成品回流的零配件中的次品共同组成。拆解回流的零件中，若经过检测则不存在次品，否则次品率维持检测前的值：

$$n_p \cdot \hat{e}_i = n_i \cdot e_i \cdot (1 - t_i) + l_i \cdot e_i' \cdot (1 - t_i') \tag{4}$$

所有用于生产的零配件的次品，将全部包含在成品的次品中，而成品的次品不论通过何种途径（检测或退货）都将面临是否拆解。因此，拆解后的各零件次品量和用于生产成品的零件的次品量有如下关系：

$$n_p \cdot \hat{e}_i \cdot d_p = n_p \cdot \hat{e_p} \cdot e_i' \cdot d_p \tag{5}$$

根据题意，可知成品的次品包括几种情况：组成零件有一个为次品则成品必为次品，以及所有零件都为合格品时成品以一定概率成为次品。可以得出成品次品率的计算公式：

$$\hat{e_p} = e_p \cdot (1 - \hat{e_1}) \cdot (1 - \hat{e_2}) + (\hat{e_1} + \hat{e_2} - \hat{e_1} \cdot \hat{(e_2)}) \tag{6}$$

分析次品流量的方程，其中含有 $e_1', e_2', \hat{e_1}, \hat{e_2}, e_p$5 个变量，共有 5 个方程。

总体流量和次品流量的所有方程合在一起，我们得到了**包含 10 个未知量的 10 个方程**。

除了流量约束之外，决策变量自身之间也可能存在一些限制性约束，这可以尽量避免选取的决策变量使问题无可行解，或直接排除某些较差的情况以减少计算量。

考虑决策变量自身的约束：

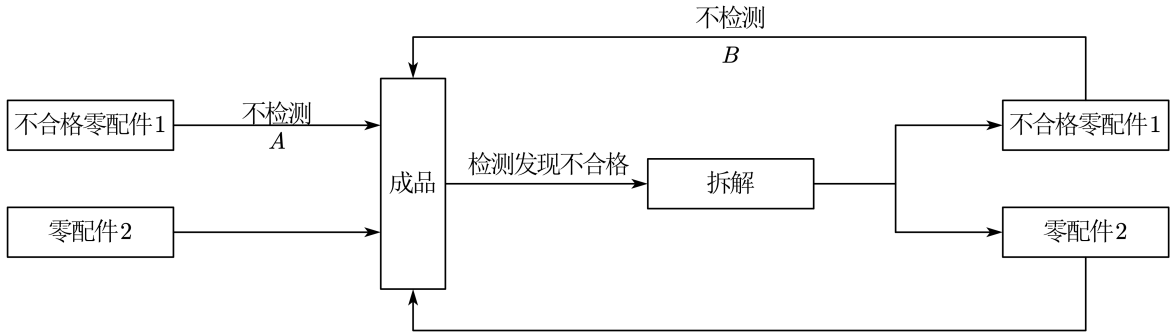

图 4 次品陷入死循环的一种情况

在此问题中，假想一种情况：零件 1 在输入产线时不进行检测，生产出的产品进行检测，遇到次品时进行拆卸再利用。此时，假若返回的零件 1 不经检测直接进入产线，将造成零件 1 的次品将永远在产线中循环而无法流出 (上图 A=0，B=0 的情况)，这将使得产线中零件 1 的次品不断累积，直到产线中的零件 1 几乎全为次品，这将导致产线的效率趋于 0。因此，在这种情况下，返回的零件 1 必须经过检验，以保证次品被淘汰出产线。也即当 $t_1 = 0, d_p = 1, t_p = 1$ 时，必须有 $t_1' = 1$。事实上，当 $t_p = 0$ 或 $d_p = 0$ 时，由于零件不会回收利用，t_p 的值对方程的解乃至目标函数并无影响，因此方便起见，将约束简化为：若 $t_1 = 0$，则 $t_1' = 1$。

现在考虑另一种情况，假如零件 1 在进入产线时首先经过检测，那么产线中的零件 1 应该都是良品，这种情况下，从成品拆解中回收的零件 1 就没有再次进行检测的必要性。对应的约束为：若 $t_1 = 1$，则 $t_1' = 0$。

对于其他的零件也是同理，于是我们得到决策变量自身之间的约束条件：

$$t_i + t_i' = 1 \tag{7}$$

5.2.3 转化为优化问题

以上的三类约束集合在一起，构成了整个产线的所有约束。为了最大化系统效益，该优化问题的目标为最小化生产单件良品的所有成本，包含零配件购买成本、检测成本、装配成本、拆解成本以及退换货造成的调换损失。其中，调换损失在不进行成品检测的情况下伴随每一件次品成品的生产而产生。将目标和约束整理成优化问题，即为：

$$
\begin{aligned}
\min_{t_1,t_2,t_1',t_2',t_p,d_p} \quad & f(t_1,t_2,t_1',t_2',t_p,d_p) \\
& = n_1 \cdot p_1 + n_2 \cdot p_2 \\
& + n_1 \cdot t_1 \cdot tc_1 + n_2 \cdot t_2 \cdot tc_2 + n_p \cdot t_p \cdot tc_p \\
& + n_p \cdot d_p \cdot \hat{e_p} \cdot (t_1' \cdot tc_1 + t_2' \cdot tc_2) \\
& + n_p \cdot ass \\
& + n_p \cdot d_p \cdot \hat{e_p} \cdot dass \\
& + n_p \cdot (1 - t_p) \cdot \hat{e_p} \cdot re \\
\text{s.t.} \quad & n_i \cdot (1 - e_i \cdot t_i) + l_i - n_p = 0 \\
& np \cdot dp \cdot \hat{e_p} \cdot (1 - e_i' \cdot t_i') - l_i = 0 \\
& np \cdot (1 - \hat{e_p}) - 1 = 0 \\
& n_p \cdot \hat{e}_i - n_i \cdot e_i \cdot (1 - t_i) + l_i \cdot e_i' \cdot (1 - t_i') = 0 \\
& n_p \cdot \hat{e}_i \cdot d_p - n_p \cdot \hat{e_p} \cdot e_i' \cdot d_p = 0 \\
& \hat{e_p} - e_p \cdot (1 - \hat{e}_1) \cdot (1 - \hat{e}_2) + (\hat{e}_1 + \hat{e}_2 - \hat{e}_1 \cdot \hat{(e_2)}) = 0 \\
& n_i, n_p \geqslant 1 \\
& l_i \geqslant 0 \\
& 0 \leqslant e_i', \hat{e}_i, \hat{p} < 1 \\
& t_i + t_i' = 1 \\
& i = 1, 2
\end{aligned}
\tag{8}
$$

5.2.4 优化问题求解

此问题中，由于决策变量数较少（6 个），总共仅有 $2^6 = 64$ 种情况（实际考虑到决策变量之间的约束关系，情况数会更少），因此直接采用枚举遍历的方法，即：对于每一组决策变量 $(t_1, t_2, t_1', t_2', t_p, d_p)$，代入上述优化问题的约束条件，求解方程组的解（根据变量数和方程数的分析，应为无解或有限个解，我们对解加以限制，如采购量 >1、次品率在 0-1 之间来获取符合实际情况的解），之后将求得的解带入目标函数，比较目标函数的值并选出最小值对应的决策变量组合。

我们对比了枚举法和遗传算法在此问题上的结果，两种方法能得到完全一致的决策变量组合，考虑枚举法找到的是全局最优解，我们认为遗传算法在此类问题中找到的最优解是值得信赖的。

在求解的具体方法上，将方程组转换为一个优化问题，最小化所有方程的均方误差，确定非决策变量的数值解，随后枚举出目标函数最小的决策变量组合（或使用遗传算法进行搜索）。

当为第一种情况，我们使用两种求解方法得到的最优结果即每销售出一个产品的最小成本均为 37.6917，此时最优的决策变量为 $(t_1 = 0, t_2 = 0, t_1' = 1, t_2' = 1, t_p = 0, d_p = 1)$，其中，$t_1(t_2)$ 为是否从刚进货的一批零件 1(2) 进行检测，$t_1'(t_2')$ 为是否对从检测不合格的成品以及从用户退回来的成品选择拆解后得到的零配件 1(2) 进行检测，t_p 为是否对成品进行检测，d_p 为是否对检测不合格的成品以及从用户退回来的成品进行拆解，决策变量为 1 则为是，为 0 则为否。

同理，对于第二种情况，我们使用两种方法得到的结果均为 46.3275，此时最优的决策变量为 $(t_1 = 1, t_2 = 1, t_1' = 0, t_2' = 0, t_p = 0, d_p = 1)$

对于第三种情况，得到的结果均为 39.7216，此时最优的决策变量为 $(t_1 = 0, t_2 = 0, t_1' = 1, t_2' = 1, t_p = 1, d_p = 1)$

对于第四种情况，得到的结果均为 43.0298，此时最优的决策变量为 $(t_1 = 1, t_2 = 1, t_1' = 0, t_2' = 0, t_p = 1, d_p = 1)$

对于第五种情况，得到的结果均为 43.2618，此时最优的决策变量为 $(t_1 = 0, t_2 = 1, t_1' = 1, t_2' = 0, t_p = 1, d_p = 1)$

对于第六种情况，得到的结果均为 34.2980，此时最优的决策变量为 $(t_1 = 0, t_2 = 0, t_1' = 1, t_2' = 1, t_p = 0, d_p = 0)$

我们将结果也记录在 problem2.xlsx 文件中。模型的合理性分析见下：

首先，两种方法求得的解完全相同，这说明遗传算法在此问题上具有适用性。其次，情况 2 中所有的次品率均高于情况 1，而情况 1 解得的进货的零配件数量 n_1、n_2 均为 1.114，情况 2 得到的值为 1.2500，这说明了为了制造出一件合格的成品在次品率更大的情况下需要进货更多的零配件，符合逻辑，而且如果零件和成品的次品率都很高，则生产出合格的产品的概率大大降低，会产生重复的装配费用，所以需要尽量对零件进行检测，所以情况 2 的 t_1 和 t_2 都为 1，而情况 1 均为 0。另外，我们发现组装成的成品 n_p 是数可以大于进货的零配件数量的，比如情况 1 的 n_p 就为 1.3276，这是因为在类似零配件 1 合格但是组装出的成品不合格的情况中，若选择拆解就会返回一些零配件 1 继续和刚进来的零配件 1 来组装，若成品的次品率足够大极有可能发生的，这也符合流水线生产产品的过程。然后我们对情况 1 和情况 3 进行对比，情况 3 的调换损失特别高，相应的，情况 1 的 t_p 为 0，而情况三的 t_p 为 1，这也符合逻辑，若调换损失远远大于成品的检测成本，则尽可能避免发生调换的情况。然后我们对情况 4 和情况 2 进行对比，发现情况 4 的零配件的检测成本很低且调换损失很高，但是两种情况装配费用相同，所以需要避免同一个零件发生多次装配以及调换的情况；相对应的，我们要尽可能的进行检测，得到的结果也符合这个逻辑。情况四的 t_1，t_2，t_p 均为 1，但是情况二的 t_1，t_2 为 1，t_p 为 0。将情况 5 和情况 1 进行对比，我们发现情况五

的零配件 1 检测成本较高，成品的检测费用更低，所以零配件 1 尽量避免检测，而是直接检测成品，所得到的结果也证明了这一点。情况 5 的 t_1 为 0，t_p 为 1，而情况 1 则相反。对于第六种情况，将其与第一种情况进行对比，发现拆解费用高，所以应尽量避免拆解，因此情况 6 的 d_p=0，而情况 1 的 d_p=1。最后，这六种情况的成本均小于市场售价 56，符合逻辑。

5.3 问题三的模型建立与求解

5.3.1 构造决策变量

1、**零配件、半成品、成品的检测**。与问题二类似，由于在各个不同环节，同种物品的次品率有所不同，因此需要将不同情况区别对待。对于零配件而言，零配件输入产线时、零配件从半成品拆解得到时（分为两种情况，分别为半成品由生产得到和拆解成品得到）三种情况下，由于中间经过了其他的生产及检测步骤，其中的次品率可能不同，因此分设三个是否检测的决策变量，分别为 t_i, t_i', t_i''。同理，对于半成品，分为生产出来之后的检测以及由成品拆解得到之后的检测，决策变量为 t_{p_j}, t'_{p_j}。最后，成品是否检测用决策变量 t_p 表示。

2、**是否拆解**。是否拆解的决策变量由检测点派生。对于不同的检测点，是否拆解的决策不一定相同。于是，半成品的两个检测点对应的是否拆解决策变量为 d_{p_j}, d'_{p_j}。而对于成品，无论是检测得到的次品还是退换货的次品，由于效益相同，是否拆解也应相同，设为 d_p。

综上，构造的决策变量包括 $t_i, t_i', t_i'', t_{p_j}, t'_{p_j}, t_p, d_{p_j}, d'_{p_j}, d_p$

由此，我们可以画出问题三一部分的产线的流程如图 5，这里我们只画了零配件 7，8 的情况。**流程图如下：**

问题三生产的具体流程和问题二类似，只不过多了一个半成品的环节，多了成品可以被拆解成半成品再被拆解成零配件的过程，相对来讲要建立的方程就要多得多，

5.3.2 根据流量构造约束方程

与问题二类似，我们通过物料平衡原理构造约束方程。事实上，问题而可以看作问题三中的子模块，一些方程的形式是类似的。**首先考虑总体流量约束**。以半成品 j 作为节点，流入它的零配件 i 等于流出它的零配件 i 的量。从流程图中可见，流入由三个部分组成，分别是新零配件的输入、生产得到的半成品 j 拆解返还、拆解得到的半成品 j 进一步拆解返还：

$$n_i \cdot (1 - e_i \cdot t_i) + l_i + l_i' = n_{p_j} \tag{9}$$

以成品作为节点，可以列出类似的方程：

$$n_{p_j} \cdot (1 - e_{p_j} \cdot t_{p_j}) + l_{p_j} = n_p \tag{10}$$

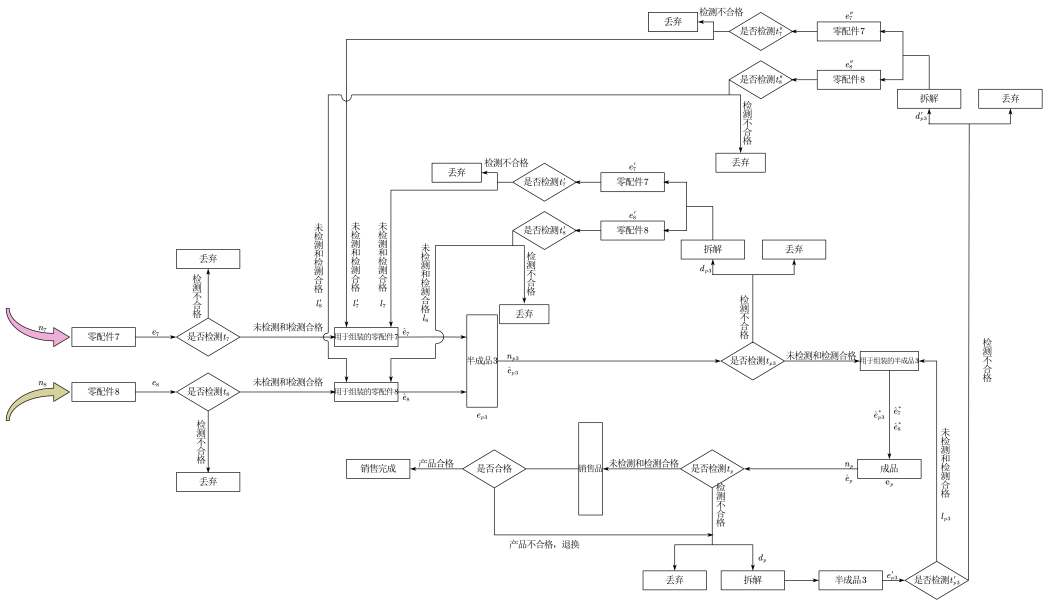

图 5 问题 3 一部分的生产流程图

以 t_i' 代表的检测点（即由生产得到的半成品拆解出的零配件的检测点）为节点，可以列出流入零配件和回收零配件之间的关系。其中，流入零配件量由半成品生产量经过半成品生产检测点回收算出：

$$n_{p_j} * t_{p_j} * d_{p_j} * \hat{e}_{p_j} * (1 - t_i' \cdot e_i') = l_i \tag{11}$$

以 t_i'' 代表的检测点（定义见变量表）作为节点，也可以列出相似的方程：

$$d_{p_j}' \cdot (n_p \cdot \hat{e}_p - l_{p_j}) \cdot (1 - t_i'' \cdot e_i'') = l_i' \tag{12}$$

以 t_{p_j} 代表的检测点（定义见变量表）作为节点，也可以列出相似的方程：

$$n_p \cdot d_p \cdot \hat{e}_p \cdot (1 - e_{p_j}' \cdot t_{p_j}') = l_{p_j} \tag{13}$$

另外，限定流出产线的良品成品为 1 个单位，于是有：

$$n_p \cdot (1 - \hat{e}_p) = 1 \tag{14}$$

分析上述总体流量方程，可知共有 $n_i, l_i, l_i', n_{p_j}, l_{p_j} n_p$ 共 31 个变量（$i = 1, 2, ..., 8; j = 1, 2, 3$）。同时出现 i, j 的方程，每条都代表 8 个方程（零配件和半成品的配对），因此也总共有 31 个方程。

接下来考虑次品流量约束。考虑半成品 j 输入和输出的零件 i 的次品量相等，其中输入的次品和前面总体流量约束相同也为三个来源，方程如下：

$$n_{p_j} \cdot \hat{e}_i = n_i \cdot e_i \cdot (1 - t_i) + l_i \cdot e_i' \cdot (1 - t_i') + l_i' \cdot e_i'' \cdot (1 - t_i'') \tag{15}$$

考虑成品输入和输出的半成品 j 的次品两相等，其中成品输入的次品包含生产出的半成品中的次品，以及拆解成品得到的半成品中流回产线的次品（问题而中已经解释过，由于含有次品的成品，无论是否检测，只要 $d_p = 1$，即拆解，都必然返回产线）。其中第二部部分需要通过成品的生产量 n_p，沿着成品的回收及拆解路径进行计算，得到方程如下：

$$\hat{e}_i \cdot n_{p_j} \cdot (1 - t_{p_j}) + \hat{e}_i^* \cdot n_p \cdot \hat{e}_p \cdot d_p \cdot (1 - t'_{p_j}) = n_p \cdot \hat{e}_i^* \tag{16}$$

以 t_{p_j} 代表的检测点为研究节点，分析其流入和流出的次品零件 i 的量，可得：

$$n_{p_j} \cdot \hat{e}_i \cdot t_{p_i} \cdot d_{p_i} = n_{p_j} \cdot t_{p_j} \cdot d_{p_j} \cdot \hat{e}_{p_j} \cdot e'_i \tag{17}$$

以 t'_{p_j} 代表的检测点为研究节点，分析其流入和流出的次品零件 i 的量，可得：

$$\hat{e}_i^* \cdot n_p \cdot d_p \cdot t'_{p_j} \cdot d'_{p_j} = n_p \cdot \hat{e}_p \cdot d_p \cdot e'_{p_j} \cdot t'_{p_j} \cdot d'_{p_j} \cdot e''_i \tag{18}$$

以成品退回节点（由于不论是检测还是退货都没有区别，因此可合为一个必然退回次品的节点）为研究节点，分析其流入和流出的次品半成品 j 的量，可得：

$$n_p \cdot \hat{e}_{p_j}^* \cdot d_p = n_p \cdot \hat{e}_p \cdot e'_{p_j} \cdot d_p \tag{19}$$

以成品生产节点为研究节点，其流入和流出的次品半成品 j 的量相等。其中流入分为生产的次品流入以及拆解返还的次品流入。可得：

$$n_p \cdot \hat{e}_{p_j}^* = n_{p_j} \cdot \hat{e}_{p_j} \cdot (1 - t_{p_j}) + l_{p_j} \cdot e'_{p_j} \cdot (1 - t'_{p_j}) \tag{20}$$

最后，根据半成品和成品的次品率的计算方法，可以写出两种生产环节产出次品率与输入次品率之间的关系。其中，半成品生产节点为：

$$\hat{e}_{p_1} = e_{p_1} \cdot (1 - \hat{e}_1)(1 - \hat{e}_2)(1 - \hat{e}_3) + (\hat{e}_1 + \hat{e}_2 + \hat{e}_3 - \hat{e}_1\hat{e}_2 - \hat{e}_1\hat{e}_3 - \hat{e}_2\hat{e}_3 + \hat{e}_1\hat{e}_2\hat{e}_3) \tag{21}$$

$$\hat{e}_{p_2} = e_{p_2} \cdot (1 - \hat{e}_4)(1 - \hat{e}_5)(1 - \hat{e}_6) + (\hat{e}_4 + \hat{e}_5 + \hat{e}_6 - \hat{e}_4\hat{e}_5 - \hat{e}_4\hat{e}_6 - \hat{e}_5\hat{e}_6 + \hat{e}_4\hat{e}_5\hat{e}_6) \tag{22}$$

$$\hat{e}_{p_3} = e_{p_3} \cdot (1 - \hat{e}_7)(1 - \hat{e}_8) + (\hat{e}_7 + \hat{e}_8 - \hat{e}_7\hat{e}_8) \tag{23}$$

成品生产节点为：

$$\hat{e}_p = e_p \cdot (1-\hat{e}_{p_1}^*)(1-\hat{e}_{p_2}^*)(1-\hat{e}_{p_3}^*) + (\hat{e}_{p_1}^* + \hat{e}_{p_2}^* + \hat{e}_{p_3}^* - \hat{e}_{p_1}^*\hat{e}_{p_2}^* - \hat{e}_{p_1}^*\hat{e}_{p_3}^* - \hat{e}_{p_2}^*\hat{e}_{p_3}^* + \hat{e}_{p_1}^*\hat{e}_{p_2}^*\hat{e}_{p_3}^*) \tag{24}$$

分析所有次品流量方程，其中包含 $\hat{e}_i, \hat{e}_i^*, e'_i, e''_i, \hat{e}_{p_j}, \hat{e}_{p_j}^*, e'_{p_j}, \hat{e}_p$ 共 42 个变量。同时，方程总数也为 42。

总体流量和次品流量的所有方程合在一起，我们得到了**包含总共 73 个未知量的 73 个方程**。

最后依然需要**考虑决策变量自身之间的约束**。事实上，该问题的本质在于避免次品没有流出只有流入的环的产生。先考虑小环：半成品 j 生产 → 半成品检测点 t_{p_j}→

零配件检测点 $t'i\rightarrow$ 半成品 j 生产，该环中，假若 t_{p_j}, d_{p_j} 都为 1，则 t_7 和 t'_7 不能同时为 0，否则次品输入该环后无法排除；也不需同时为 1，因为若零配件 i 在输入时已经经过检验，则后续不需再进行检验。并且，若 t_{p_j}, d_{p_j} 有 0，则次品零件 i 必会从 d_{p_j} 处流出环，因此检测点 t'_i 可以任意设置。于是我们得到简化的约束：

$$t_i + t'_i = 1 \tag{25}$$

然后我们考虑大环：半成品 j 生产 → 半成品检测点 $t_{p_j}\rightarrow$ 成品生产 → 成品退还与拆解 → 半成品检测点 $t'_{p_j}\rightarrow$ 零配件检测点 $t''_i\rightarrow$ 半成品 j 生产。与上述分析类似，假若 t_{p_j} 为 0，d_p, t'_{p_j}, d'_{p_j} 都为 1，为了避免次品零件 i 有流入无流出的情况，t_i 和 t''_i 不能同时为 0，并且不需同时为 1。而在其他情况下，次品零配件 i 必然在 t''_i 节点之前流出环，因此 t''_i 可以任意设置。于是我们得到简化的约束：

$$t_i + t''_i = 1 \tag{26}$$

最后，考虑关于半成品 j 的次品的环：成品生产 → 成品退还与拆解 → 半成品检测点 $t'_{p_j}\rightarrow$ 成品生产。采用与前面一样的分析方法，可以得出:

$$t_{p_j} + t'_{p_j} = 1 \tag{27}$$

5.4 转化为优化问题

至此，我们已经找出了系统内的全部约束。将以上三类约束合并在一起，可以将问题转化为一个约束问题。与问题二相同，目标仍然是最小化单件良品成品的生产成本。

$$
\begin{aligned}
\min_{t_i, t_i', t_i'', t_{p_j}, t_{p_j}', t_p, d_{p_j}, d_{p_j}', d_p} & f(t_i, t_i', t_i'', t_{p_j}, t_{p_j}', t_p, d_{p_j}, d_{p_j}', d_p) \\
& = \sum_{i=1}^{8} n_i \cdot (p_i + t_i \cdot tc_i) \\
& + \sum_{i,j} tc_i \cdot (t_i' \cdot n_{p_j} \cdot t_{p_j} \cdot d_{p_j} \cdot \hat{e}_{p_j} + t_i'' \cdot n_p \cdot \hat{e}_p \cdot d_p \cdot e_{p_j}' \cdot t_{p_j}' \cdot d_{p_j}') \\
& + \sum_{j=1}^{3} tc_{p_j} \cdot (n_{p_j} \cdot t_{p_j} + n_p \cdot d_p \cdot \hat{e}_p \cdot t_{p_j}') \\
& + \sum_{j=1}^{3} ass_{p_j} \cdot n_{p_j} \\
& + \sum_{j=1}^{3} dass_{p_j} \cdot (n_{p_j} \cdot t_{p_j} \cdot d_{p_j} \cdot \hat{e}_{p_j} + n_p \cdot \hat{e}_p \cdot d_p \cdot e_{p_j}' \cdot t_{p_j}') \\
& + n_p \cdot t_p \cdot tc_p + n_p \cdot ass_p + n_p \cdot d_p \cdot \hat{e}_p \cdot dass_p \\
& + n_p \cdot (1 - t_p) \cdot \hat{e}_p \cdot re \\
\text{s.t.} \quad & n_i \cdot (1 - e_i \cdot t_i) + l_i + l_i' - n_{p_j} = 0 \\
& n_{p_j} \cdot (1 - e_{p_j} \cdot t_{p_j}) + l_{p_j} - n_p = 0 \\
& n_{p_j} * t_{p_j} * d_{p_j} * \hat{e}_{p_j} * (1 - t_i' \cdot e_i') - l_i = 0 \\
& d_{p_j}' \cdot (n_p \cdot \hat{e}_p - l_{p_j}) \cdot (1 - t_i'' \cdot e_i'') - l_i' = 0 \\
& n_p \cdot d_p \cdot \hat{e}_p \cdot (1 - e_{p_j}' \cdot t_{p_j}') - l_{p_j} = 0 \\
& n_p \cdot (1 - \hat{e}_p) - 1 = 0 \\
& n_{p_j} \cdot \hat{e}_i - n_i \cdot e_i \cdot (1 - t_i) - l_i \cdot e_i' \cdot (1 - t_i') - l_i' \cdot e_i'' \cdot (1 - t_i'') = 0 \\
& \hat{e}_i \cdot n_{p_j} \cdot (1 - t_{p_j}) + \hat{e}_i^* \cdot n_p \cdot \hat{e}_p \cdot d_p \cdot (1 - t_{p_j}') - n_p \cdot \hat{e}_i^* = 0 \\
& n_{p_j} \cdot \hat{e}_i \cdot t_{p_i} \cdot d_{p_i} - n_{p_j} \cdot t_{p_j} \cdot d_{p_j} \cdot \hat{e}_{p_j} \cdot e_i' = 0 \\
& \hat{e}_i^* \cdot n_p \cdot d_p \cdot t_{p_j}' \cdot d_{p_j}' - n_p \cdot \hat{e}_p \cdot d_p \cdot e_{p_j}' \cdot t_{p_j}' \cdot d_{p_j}' \cdot e_i'' = 0 \\
& n_p \cdot \hat{e}_{p_j}^* \cdot d_p - n_p \cdot \hat{e}_p \cdot e_{p_j}' \cdot d_p = 0 \\
& n_p \cdot \hat{e}_{p_j}^* - n_{p_j} \cdot \hat{e}_{p_j} \cdot (1 - t_{p_j}) - l_{p_j} \cdot e_{p_j}' \cdot (1 - t_{p_j}') = 0 \\
& \hat{e}_{p_1} - e_{p_1} \cdot (1 - \hat{e}_1)(1 - \hat{e}_2)(1 - \hat{e}_3) - \\
& \qquad (\hat{e}_1 + \hat{e}_2 + \hat{e}_3 - \hat{e}_1\hat{e}_2 - \hat{e}_1\hat{e}_3 - \hat{e}_2\hat{e}_3 + \hat{e}_1\hat{e}_2\hat{e}_3) = 0 \\
& \hat{e}_{p_2} - e_{p_2} \cdot (1 - \hat{e}_4)(1 - \hat{e}_5)(1 - \hat{e}_6) - \\
& \qquad (\hat{e}_4 + \hat{e}_5 + \hat{e}_6 - \hat{e}_4\hat{e}_5 - \hat{e}_4\hat{e}_6 - \hat{e}_5\hat{e}_6 + \hat{e}_4\hat{e}_5\hat{e}_6) = 0 \\
& \hat{e}_{p_3} - e_{p_3} \cdot (1 - \hat{e}_7)(1 - \hat{e}_8) - (\hat{e}_7 + \hat{e}_8 - \hat{e}_7\hat{e}_8) = 0 \\
& \hat{e}_p - e_p \cdot (1 - \hat{e}_{p_1}^*)(1 - \hat{e}_{p_2}^*)(1 - \hat{e}_{p_3}^*) - \\
& \qquad (\hat{e}_{p_1}^* + \hat{e}_{p_2}^* + \hat{e}_{p_3}^* - \hat{e}_{p_1}^*\hat{e}_{p_2}^* - \hat{e}_{p_1}^*\hat{e}_{p_3}^* - \hat{e}_{p_2}^*\hat{e}_{p_3}^* + \hat{e}_{p_1}^*\hat{e}_{p_2}^*\hat{e}_{p_3}^*) \\
& n_i, n_{p_j}, n_p \geqslant 1 \\
& l_i, l_i', l_{p_j} \geqslant 0 \\
& 0 \leqslant \hat{e}_i, \hat{e}_i^*, e_i', e_i'', \hat{e}_{p_j}, \hat{e}_{p_j}^*, e_{p_j}', \hat{e}_p < 1 \\
& t_i + t_i' = 1, t_i + t_i'' = 1, t_{p_j} + t_{p-j}' = 1
\end{aligned}
\tag{28}
$$

5.5 优化问题求解

此问题的决策变量为 $t_i, t_i', t_i'', t_{p_j}, t_{p_j}', t_p, d_{p_j}, d_{p_j}', d_p$，共有 8×3+3×2+1+3×2+1 共 38 个决策变量。考虑到决策变量自身之间的约束，可以减少实际决策变量的数量：将 t_i', t_i'' 用 $1-t_i$ 替代，t_{p_j}' 用 $1-t_{p_j}$ 替代。于是，实际独立的决策变量为 $t_i, t_{p_j}, t_{p_j}', t_p, d_{p_j}, d_p$ 共 19 个，这对应 $2^{19} \approx 5.24\times10^5$ 种取值情况，且由于确定非决策变量的方程数大大增加，单次求解速度也将变慢，通过穷举法寻找最优决策变量组合的方法不再适用，因此考虑采用启发式的搜索方法。

不难发现，当我们显式求解出决策变量与非决策变量的约束方程，规划问题实际上转变为了一个最小化成本的优化问题，因此我们认为采用遗传算法进行优化是一个理想的选择。

遗传算法在每次迭代中使用适应度函数对个体（候选解）进行评估，适应度得分更高的个体代表了更好的解，更有可能被选择繁殖，在优化过程中留下其性状。为了创建新个体，遗传算法会将双亲个体的染色体互换，以创建代表后代的两个新染色体，同时，为了避免陷入局部最优，遗传算法引入了突变，将新模式引入染色体，以鼓励算法在解空间的未知区域进行搜索。对于整数决策变量的变异，我们保证每次变异为从 0 变为 1 或从 1 变为 0 即可。

遗传算法流程图如图 6

通过使用遗传算法来求解, 我们得到的最优的结果是每销售出一个产品的最小成本为 143.0543，各个决策变量的值保存到了 problem3.xlsx。

每一代的最优个体目标函数（成本）如图 7

在对模型结果及最优个体的成本图进行分析时，我们观察到成本曲线随着迭代的进行逐步下降，这表明求解过程正逐渐收敛。最终的典型均方误差（MSE）为 1×10^{-6} 数量级，说明基本满足约束。

此外，我们发现调换损失 40 相较于检测成本显著更高，因此推测最优策略应设置 $t_p = 1$，这一结论也在结果中得到了验证。并且每销售一件合格品所需的每种零配件数大致都接近 1，这一结果符合逻辑。

为了更好地验证模型的合理性，我们决定对情况 1 的数据进行修改，并与原来的情况 1 进行对比。我们进行了以下几种修改：

1. 将零配件 1 的检测成本设置为 40，可以推测出因为检测成本的提高导致零配件 1 不进行检测，交给遗传算法来求解，发现 t_1 变成了 0，而原来的情况 1 中的 $t_1 = 1$，说明模型做出了正确的决策。

2. 将半成品 1 的拆卸成本设置为 40，可以推测出因为检测成本的提高导致半成品 1 不进行拆卸，交给遗传算法来求解，发现 d_{p1} 变成了 0，而原来的情况 1 中的 $d_{p1} = 1$，说明模型做出了正确的决策。

3. 将成品的调换损失设置为 1，可以推测出因为不合格的成品被调换的价格比对

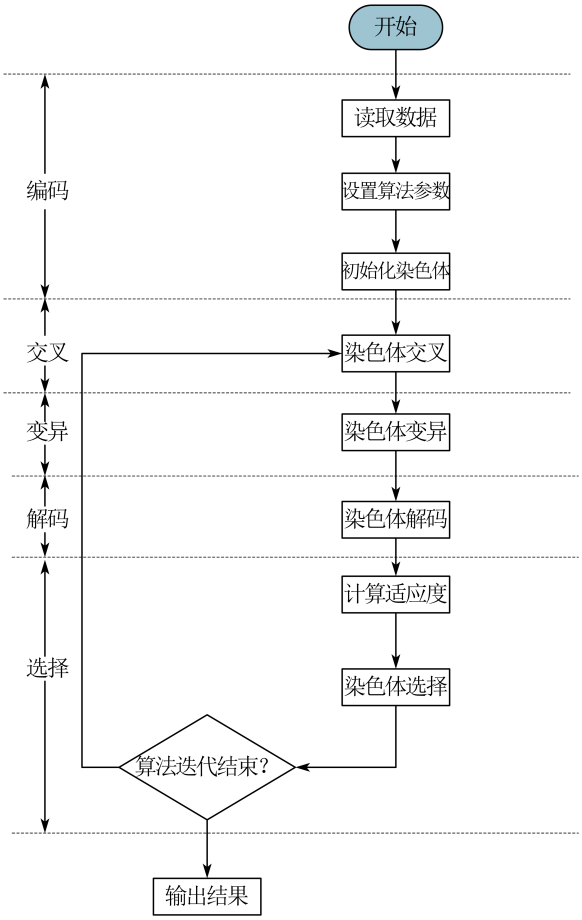

图 6 遗传算法流程图

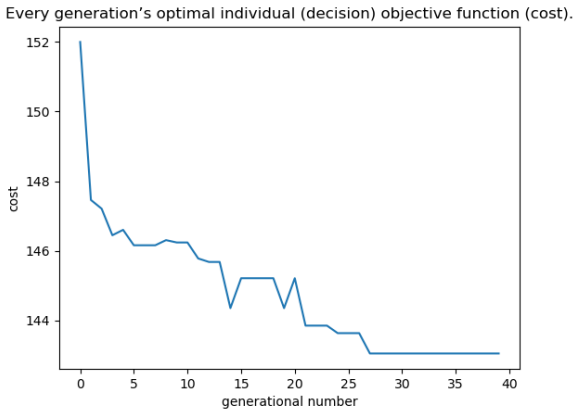

图 7 每一代最优个体的成本图

成品进行检测的费用低得多的，这就相当于让花费更少的钱让用户来帮我们进行检测，所以可以推测出 $t_p = 0$。交给遗传算法来求解，发现果然 t_p 变成了 0，而原来的情况 1 中的 $t_p = 1$，说明模型做出了正确的决策。

综上所述，模型能够准确判断是否进行检测和拆卸。最后，最小成本低于市场售价 200，表明企业盈利，这符合实际情况。因此，我们可以确认模型得出的结果是合

理的。

5.6 问题四的模型建立与求解

问题四的求解思路如图 8

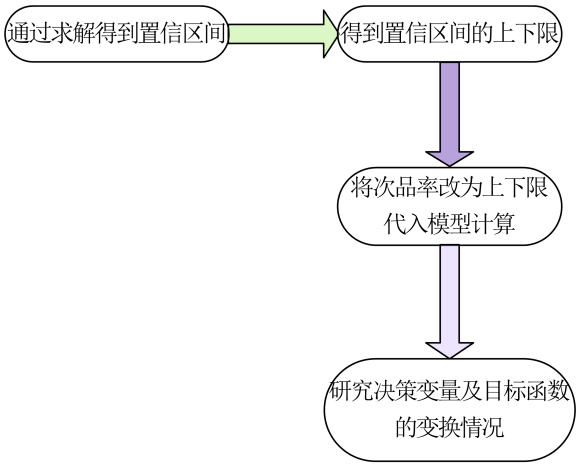

图 8 思路 4

5.6.1 实验设置与结果猜想

按照问题分析，我们首先需要构造置信区间。设定置信度为 95%。

我们假设：估计次品率时，抽取的样本大小为 100。则检测次品率为 10% 时，意味着其中检测出了 10 个次品，由此计算得出次品率为 10% 的结论。设真实次品率为 p，由于问题是单边的，p 的 95% 置信区间应由下列两个不等式解出：

$$\sum_{i=10}^{100} \binom{100}{i} \cdot p^i \cdot (1-p)^{100-i} \geqslant 0.05 \tag{29}$$

$$\sum_{i=0}^{10} \binom{100}{i} \cdot p^i \cdot (1-p)^{100-i} \geqslant 0.05 \tag{30}$$

得到 10% 次品率的 95% CI 约为 $[0.055, 0.151]$。同样的方法可得，20% 和 5% 次品率的 CI 分别为 $[0.010, 0.266]$ 和 $[0.020, 0.100]$

具体方法上，每次只研究一个次品率变量的影响，将选定研究的次品率分别设置为置信区间的上限和下限，其余次品率维持原值，代入模型进行计算，研究决策变量的解以及目标函数的变化情况。根据猜想，解的变化应该有几种可以预见的典型情形：

1、原本的最优解中，零部件 i 在输入产线时不做检测。而若将零部件 i 的次品率设置为置信上界时，它的值增大，导致不检测所造成的后续损失增大。在这种情况下，模型新的最优解中，零部件 i 在输入产线时可能变成需要检测。这种情况应当伴随着目标函数的增大（即成本上升）。

2、与 1 相反的情况。原本的最优解中，零部件 i 在输入产线时需要检测。而将零部件 i 的次品率设置为置信下界时，它的值减小，导致检测的必要性被减弱（因为对良品的检测是无意义的成本消耗）。这种情况下，模型新的最优解中，零部件 i 在输入产线时可能变成不需要检测。这种情况应当但随着目标函数的减小（即成本降低）。

3、对于模型三半成品和成品而言，也应当有与零件相似的规律。此外，半成品的是否检测，和零件的是否检测之间可能也存在一定的关系。每次改变次品率参数时，可能有多个决策变量一同变化。

4、模型不受影响的情况，例如某个零配件原本需要检测，如果输入产线的该零配件的次品率升高，则它应当依然要被检测，而其余解应当不受影响。

5.6.2 实验结果分析

我们在问题二和问题三的模型上分别进行了实验（实验结果见附录1），通过实验验证了以上猜测。由于求解一次模型耗时较久（10 分钟左右），我们没有研究所有参数的所有变化的影响，而是挑选了一些我们预期会对模型最优解产生典型影响的参数进行研究，结果如下：

问题二：问题二的模型较为简单，我们在情况二上测试了上述第四种猜想，即不影响解的情形：将零件一的次品率由 20% 调整为置信上界 26.6%，得到的决策变量的新解与原始解完全相同，而目标函数（单件合格成品的综合成本）由 46.3 增加到 46.9。结果符合预期。

问题三：

1、将半成品 1 的次品率设置为置信下界 0.055。注意到，原始解中，零配件 1 在输入产线时需要检测 ($t_{p_1} = 1$)，最优目标函数为 143.05。修改参数后，新的解中，t_{p_1} 变为 0，意味着生产出的半成品 1 不再立即进行检测。同时，最优目标函数有所降低，变为 141.98。这符合预期。观察其他决策变量的变化情况，发现 t_3 由 0 变为 1，即零部件 1 在输入产线时不检测，这可能时由于半成品 1 在生产后不再经过检测，所以提高了其组成零件的检测必要性；另外，注意到 t_5 由 1 变为 0，即零部件 5 在输入产线时不经过检测，而 t_{p_2} 由 0 变为 1，即半成品 2 在生产之后需要检测，这也是合理的，因为零部件 5 输入时不检测会导致半成品 2 次品率的上升，从而提高检测的必要性。

2、将成品的次品率设置为置信下界 0.055。原始解中，$t_p = 1$，即成品需要检测。而新的模型解中，$t_p = 0$，即成品不需要检测，这也是符合预期的。同时目标函数也由原始的 143.05 降低为 142.90

3、将零配件 3 的次品率设置为置信上界 0.151。t_i 原本为不检测，新解中仍然为不检测。其余的决策变量也几乎保持不变。这说明模型对零配件 3 的次品率上升较不敏感。

5.7 问题四的总结

在这个问题中，我们首先计算了真实参数的置信区间，然后从敏感性分析的角度，研究了参数取到置信区间边界时，对模型产生的影响。实验的结果较好地证实了我们的猜想，这是模型真实有效性的一种印证。另外，由于遗传算法的性质，所取得的解可能并非最优，且由于方程组求解所使用的数值方法，对于变量值和目标函数值的计算也可能存在一定误差，因此对于模型较不敏感的变量，不一定总能得出符合预期的结果（比如检测成本极低的零配件，其输入产线的次品率 e_i 不一定能完美反映在决策变量中）。

六、模型的评价及推广

6.1 模型的优点

1. 对于问题一，在进行假设后建立了严格的统计检验模型，能够在满足置信度要求的同时使抽样次数不至于过高。
2. 对于问题二、三，首先基于物料平衡原则建立了准确的生产流程模型，并得到最小化成本的优化问题，随后采用分布优化的思想方法，对于难以求解的约束方程，通过数值方法得到数值解，再对决策变量进行枚举优化或遗传算法优化。基于此，我们能精准了解生产过程的每一个状态，并得到极为精准的最优决策。
3. 对于问题四，通过置信区间的求解，我们能够基于生产流程模型进行各个决策变量的敏感性分析，得到对次品率高敏感和不敏感的决策变量，提高了模型的鲁棒性，并减少了模型对抽样检测的依赖。

6.2 模型的缺点

1. 遗传算法不一定能找到全局最优点，但根据问题二的分析，在问题三中遗传算法的解依然有很高的准确性。
2. 对于问题三，求解一次需要花费十几分钟，时间代价较高。
3. 求解约束方程依赖于数值方法，若问题规模进一步扩大，生产流程环节增多，约束方程不一定能解出正确的数值解，且遗传算法的优化难度也会增大。

6.3 模型的推广

1. 本文基于生产流程决策的应用场景，构建了零配件组装过程的决策模型，对不同的实际生产工况进行了讨论，验证了本模型的合理性与准确性，本模型在我国智能制造领域可以发挥重要应用。
2. 本模型可以应用于任意需要决策的工程类问题。

参考文献

[1] 李晓叶, 张京丽, 于妍. 动态图表展现遗传算法基本原理 [J]. 电脑编程技巧与维护,2017,(12):80-81+94.DOI:10.16184/j.cnki.comprg.2017.12.028.

[2] 余雨薇. 面向节能的 S 公司 PCB 插件混流生产车间排产优化研究 [D]. 浙江大学,2023. DOI:10.27461/d.cnki.gzjdx.2023.001351.

[3] 许靖菡.Q 公司啤酒包装车间调度优化研究 [D]. 西安理工大学,2023. DOI:10.27398/d.cnki.gxalu.2023.001980.

[4] 李明鸣. 考虑物料平衡的 N 公司 J 生产线流程优化研究 [D]. 中国矿业大学,2023. DOI:10.27623/d.cnki.gzkyu.2023.002946.

附　录

附录 1：支撑材料目录

文件名	内容
problem1_1.xlsx	问题一的情形一对应的概率表
problem1_2.xlsx	问题一的情形二对应的概率表
problem2.xlsx	问题二的决策结果及综合成本
problem3.xlsx	问题三的决策结果及综合成本
problem4_case1.xlsx	问题四中重做问题二的实验
problem4_case2.xlsx	问题四中重做问题三的实验 1
problem4_case3.xlsx	问题四中重做问题三的实验 2
problem4_case4.xlsx	问题四中重做问题三的实验 3
problem1.py	问题一绘制概率列表的程序
problem2_1.py	问题二的遍历求解算法
problem2_2.py	问题二的遗传求解算法
problem3.py	问题三的遗传求解算法
problem4_CI.py	问题四的置信区间计算
problem4_re2.py	按照问题四的假设重做问题二
problem4_re3.py	按照问题四的假设重做问题三

附录 2：问题一的 Python 代码

```python
import pandas as pd
import numpy as np
import os
from math import comb

def get_prob(N, n, p = 0.1):
  """
  获取一批大小为N的产品中有n个次品的概率
  N: 产品数量
  n: 次品数量
  p: 产品为次品的概率，默认为0.1
  """
  prob = comb(N, n) * (p ** n) * ((1 - p) ** (N - n))
  return prob
```

```python
def generate_matrix(t, sign):
  # 保存Pi(X=j)的概率
  arr = np.zeros((t, t))
  for i in range(1, t):
    for j in range(i + 1):
      arr[i, j] = get_prob(i, j, 0.1)

  cumulative_arr = arr.copy()
  if sign == 1:
    # H0: 零配件次品率低于标称值, H1: 零配件次品率高于标称值
    for i in range(1, t):
      for j in range(i - 1, -1, -1): # 第一小问，从右往左累加概率得到Pi(X
          >=j)
        cumulative_arr[i, j] += cumulative_arr[i, j + 1]

  elif sign == 2:
    # H0: 零配件次品率高于标称值, H1: 零配件次品率低于标称值
    for i in range(1, t):
      for j in range(1, i + 1): # 第二小问，从左往右累加概率得到Pi(X<=j)
        cumulative_arr[i, j] += cumulative_arr[i, j - 1]

  # 转换为dataframe
  df_cumulative = pd.DataFrame(cumulative_arr)
  df_cumulative.columns = [f'{i}' for i in range(t)]
  df_cumulative.index = [f'{i}' for i in range(t)]

  # 保存至excel中
  mkdir = 'result'
  if not os.path.exists(mkdir):
    os.makedirs(mkdir)

  file_name = f"./result/problem1_{sign}.xlsx"
  df_cumulative.to_excel(file_name, index=True)

  print(f"DataFrame has been saved to {file_name}")

  print(df_cumulative)

generate_matrix(10, 1) # 生成情形1结果
generate_matrix(30, 2) # 生成情形2结果
```

附录 3：问题二的枚举算法的 Python 代码

```python
# 问题2非线性0-1整数规划的枚举求解
# 6个决策变量分别置为0或1，总共是2^6种情况，因此可以采用枚举进行求解

import itertools
from scipy.optimize import minimize

# 定义问题中的常数
e1, e2, ep = 0.2, 0.2, 0.2 # 零件与成品的次品率
p1, p2 = 4, 18 # 零件各自的采购价
tc1, tc2, tcp = 2, 3, 3 # 零件与成品的检测成本
ass, dass = 6, 5 # 组装成本与拆解成本
re = 6 # 调换成本

# 定义目标函数，即成本
def objective(solution, t1, t2, t1_, t2_, tp, dp):
  n1, n2, l1, l2, np, e1h, e2h, e1_, e2_, eph = solution
  obj = n1 * p1 + n2 * p2 + \
    n1 * t1 * tc1 + n2 * t2 * tc2 + np * tp * tcp + \
    np * dp * eph * (t1_ * tc1 + t2_ * tc2) + \
    np * ass + \
    np * dp * eph * dass + \
    np * (1 - tp) * eph * re
  return obj

# 定义 MSE 方程，用于求解非决策变量
def mse(vars, t1, t2, t1_, t2_, tp, dp):
  n1, n2, l1, l2, np, e1h, e2h, e1_, e2_, eph = vars

  # 流量约束方程
  eq1 = n1 * (1 - e1 * t1) + l1 - np
  eq2 = n2 * (1 - e2 * t2) + l2 - np
  eq3 = np * dp * eph * (1 - e1_ * t1_) - l1
  eq4 = np * dp * eph * (1 - e2_ * t2_) - l2
  eq5 = np * (1 - eph) - 1

  # 次品率约束方程
  eq6 = np * e1h - n1 * e1 * (1 - t1) - l1 * e1_ * (1 - t1_)
  eq7 = np * e2h - n2 * e2 * (1 - t2) - l2 * e2_ * (1 - t2_)
  eq8 = e1h * dp - eph * e1_ * dp
```

```
    eq9 = e2h * dp - eph * e2_ * dp
    eq10 = eph - ep * (1 - e1h) * (1 - e2h) - (e1h + e2h - e1h * e2h)

    return sum(x**2 for x in [eq1, eq2, eq3, eq4, eq5, eq6, eq7, eq8,
        eq9, eq10])/10

# 开始枚举
combinations = list(itertools.product([0, 1], repeat=6))
initial_guess = [0.5 for _ in range(10)]

min_obj = 0
i = 0
for choice in combinations:
    t1, t2, t1_, t2_, tp, dp = choice
    # 跳过不满足决策变量约束的解，只枚举满足约束的解
    if (t1 + t1_ - 1) != 0 or (t2 + t2_ - 1) != 0:
        continue
    # 调用 minimize，并传入约束条件列表
    solution = minimize(lambda vars: mse(vars, t1, t2, t1_, t2_, tp,
        dp),
            initial_guess,
            options={'maxiter': 5000},
            tol=1e-6,
            bounds=[(0, None) for _ in range(5)] + [(0, 1) for _ in
                range(5)])

    # 获取结果并检查
    print(i)
    print(solution.message)
    if solution.success:
        vars_solution = solution.x # 从优化结果中提取解
        print(f't1: {t1}, t2: {t2}, t1_: {t1_}, t2_: {t2_}, tp: {tp}, dp
            : {dp}')
        print(f'n1: {vars_solution[0]}, n2: {vars_solution[1]}, l1: {
            vars_solution[2]}, l2: {vars_solution[3]}, np: {vars_solution
            [4]}, e1h: {vars_solution[5]}, e2h: {vars_solution[6]}, e1_:
            {vars_solution[7]}, e2_: {vars_solution[8]}, eph: {
            vars_solution[9]}')
        print(f'MSE: {mse(vars_solution, t1, t2, t1_, t2_, tp, dp)}')
```

```
    obj = objective(vars_solution, t1, t2, t1_, t2_, tp, dp) # 计算目
        标函数
    print(f'Objective: {obj}')

    # 更新最优解
    if i == 0 or obj < min_obj:
      min_obj = obj
      best_choice = choice
      best_solution = vars_solution
  i += 1

# 输出最优结果
print('\n最优结果: ')
print(f't1: {best_choice[0]}, t2: {best_choice[1]}, t1_: {
    best_choice[2]}, t2_: {best_choice[3]}, tp: {best_choice[4]}, dp
    : {best_choice[5]}')
print(f'min_obj: {min_obj}')
print(f'n1: {best_solution[0]}, n2: {best_solution[1]}, l1: {
    best_solution[2]}, l2: {best_solution[3]}, np: {best_solution
    [4]}, e1h: {best_solution[5]}, e2h: {best_solution[6]}, e1_: {
    best_solution[7]}, e2_: {best_solution[8]}, eph: {best_solution
    [9]}')

# 枚举法与遗传算法在第二问的解完全一致，故不保存excel结果
# 可以自行改变情况常数进行求解复现
```

附录 4：问题二的遗传算法的 Python 代码

```
# 问题2非线性0-1整数规划的遗传算法求解

import numpy as np
from scipy.optimize import minimize
from deap import base, creator, tools, algorithms
import pandas as pd
# import matplotlib.pyplot as plt
import openpyxl

# 定义问题中的常数
# 代码重构时考虑了六种情况，将常数保存至params[i]中，写入for循环
'''e1, e2, ep = 0.1, 0.2, 0.1 # 零件与成品的次品率
p1, p2 = 4, 18 # 零件各自的采购价
```

```
tc1, tc2, tcp = 8, 1, 2 # 零件与成品的检测成本
ass, dass = 6, 40 # 组装成本与拆解成本
re = 6 # 调换成本'''

# 创建遗传算法的适应度函数，目标是最小化问题
creator.create("FitnessMin", base.Fitness, weights=(-1.0,))
creator.create("Individual", list, fitness=creator.FitnessMin)

# 初始化种群的个体，整数变量(0或1)
def init_individual():
  # 整数变量部分：t1, t2, tp, dp, tn1, tn2 (0或1)
  int_vars = np.random.randint(0, 2, 4).tolist()
  return creator.Individual(int_vars)

def objective(solution, t1, t2, t1_, t2_, tp, dp, params): # 目标函
    数，即成本
  n1, n2, l1, l2, np, e1h, e2h, e1_, e2_, eph = solution
  p1, p2 = params[0:2]
  tc1, tc2, tcp = params[2:5]
  ass, dass = params[5:7]
  re = params[7]
  obj = n1 * p1 + n2 * p2 + \
      n1 * t1 * tc1 + n2 * t2 * tc2 + np * tp * tcp + \
      np * dp * eph * (t1_ * tc1 + t2_ * tc2) + \
      np * ass + \
      np * dp * eph * dass + \
      np * (1 - tp) * eph * re
  return obj

def mse(vars, t1, t2, t1_, t2_, tp, dp, params): # 计算约束方程的均方
    误差
  n1, n2, l1, l2, np, e1h, e2h, e1_, e2_, eph = vars # 采购流量，拆解
      流量，总流量，以及次品率
  e1, e2, ep = params
  # 流量约束
  eq1 = n1 * (1 - e1 * t1) + l1 - np
  eq2 = n2 * (1 - e2 * t2) + l2 - np
  eq3 = np * dp * eph * (1 - e1_ * t1_) - l1
  eq4 = np * dp * eph * (1 - e2_ * t2_) - l2
  eq5 = np * (1 - eph) - 1
```

```
    # 次品率约束
    eq6 = np * e1h - n1 * e1 * (1 - t1) - l1 * e1_ * (1 - t1_)
    eq7 = np * e2h - n2 * e2 * (1 - t2) - l2 * e2_ * (1 - t2_)
    eq8 = e1h * dp - eph * e1_ * dp
    eq9 = e2h * dp - eph * e2_ * dp
    eq10 = eph - ep * (1 - e1h) * (1 - e2h) - (e1h + e2h - e1h * e2h)

    return sum(x**2 for x in [eq1, eq2, eq3, eq4, eq5, eq6, eq7, eq8,
        eq9, eq10])/10

# 评估目标函数的值
def evaluate(individual, params):
    t1, t2, tp, dp = individual
    t1_, t2_ = 1 - t1, 1 - t2
    # 最小化均方误差mse，以此解得各个非决策变量的数值解
    initial_guess = [0.5 for _ in range(10)]
    solution = minimize(lambda vars: mse(vars, t1, t2, t1_, t2_, tp,
        dp, params[0:3]),
            initial_guess,
            options={'maxiter': 3000},
            tol=1e-6,
            bounds=[(0, None) for _ in range(5)] +\
            [(0, 1) for _ in range(5)]) #流量约束为>0, 次品率约束为0-1之间
                的连续值

    if solution.success:
        vars_solution = solution.x
        print(f't1: {t1}, t2: {t2}, t1_: {t1_}, t2_: {t2_}, tp: {tp}, dp
            : {dp}')
        print(f'n1: {vars_solution[0]}, n2: {vars_solution[1]}, l1: {
            vars_solution[2]}, l2: {vars_solution[3]}, np: {vars_solution
            [4]}, e1h: {vars_solution[5]}, e2h: {vars_solution[6]}, e1_:
            {vars_solution[7]}, e2_: {vars_solution[8]}, eph: {
            vars_solution[9]}')
        print(f'MSE: {mse(vars_solution, t1, t2, t1_, t2_, tp, dp,
            params[0:3])}')

        # 计算目标函数的值
        objective_value = objective(vars_solution, t1, t2, t1_, t2_, tp,
            dp, params[3:11])
```

```
    print(f'Objective value: {objective_value}')
    return objective_value,

  else:
    return float('inf'), # 如果优化失败，返回无穷大，快速淘汰此个体

# 注册遗传算法的主要操作
toolbox = base.Toolbox()
toolbox.register("individual", init_individual)
toolbox.register("population", tools.initRepeat, list, toolbox.
    individual)
toolbox.register("mate", tools.cxUniform, indpb=0.5) # 使用均匀交叉

# 为整数变量定义变异操作，确保变异后值仍为0或1
def mutate_integer(individual, indpb):
  for i in range(4):
    if np.random.random() < indpb:
      individual[i] = 1 - individual[i] # 0变1，1变0
  return individual,

# 注册整数的变异
toolbox.register("mutate", mutate_integer, indpb=0.2)

# 注册选择操作（锦标赛选择）
toolbox.register("select", tools.selTournament, tournsize=3)

# 定义遗传算法的执行步骤
def run_ga(params):

  toolbox.register("evaluate", evaluate, params=params)

  # 设置种群数为100
  population = toolbox.population(n=100)
  # 初始化一个列表来存储每一代的最低值
  min_values_per_generation = []

  # 进行5代进化
  for gen in range(5):
    offspring = algorithms.varAnd(population, toolbox, cxpb=0.7,
        mutpb=0.2)
    fits = toolbox.map(toolbox.evaluate, offspring)
```

```
    for ind, fit in zip(offspring, fits):
      ind.fitness.values = fit
    population = toolbox.select(offspring, k=len(population))
    now_best = tools.selBest(population, k=1)[0]

    # 记录这一代的最低值
    min_value = toolbox.evaluate(now_best)
    min_values_per_generation.append(min_value)
    print(f"Generation {gen}: Min value = {min_value}")

  return population, min_values_per_generation

params = []
params.append([0.1, 0.1, 0.1, 4, 18, 2, 3, 3, 6, 5, 6])
params.append([0.2, 0.2, 0.2, 4, 18, 2, 3, 3, 6, 5, 6])
params.append([0.1, 0.1, 0.1, 4, 18, 2, 3, 3, 6, 5, 30])
params.append([0.2, 0.2, 0.2, 4, 18, 1, 1, 2, 6, 5, 30])
params.append([0.1, 0.2, 0.1, 4, 18, 8, 1, 2, 6, 5, 10])
params.append([0.05, 0.05, 0.05, 4, 18, 2, 3, 3, 6, 40, 10])

# 保存位置
file_path = './result/problem2.xlsx'

book = openpyxl.Workbook()
book.save(file_path)

for i in range(6):
  # 执行遗传算法
  final_population, min_values = run_ga(params[i])

  # 输出最优个体和其目标值
  best_individual = tools.selBest(final_population, k=1)[0]
  print("Best individual:", best_individual)
  print("Objective value:", toolbox.evaluate(best_individual))

  # 将结果保存至excel中
  t = best_individual[:2]
  tp = best_individual[2]
  dp = best_individual[3]
  t_ = [1-t[0], 1-t[1]]
```

```
    result_1 = pd.DataFrame({'t': t, 't\'': t_}, index=range(1, 3))
    result_2 = pd.DataFrame({'tp': tp, 'dp': dp}, index=[0])
    result = pd.concat([result_1, result_2])

    with pd.ExcelWriter(file_path, engine='openpyxl', mode='a',
        if_sheet_exists='replace') as writer:
      result.to_excel(writer, sheet_name=f'情况{i+1}')


    # 情况较少，易于优化，故不打印优化曲线
    '''plt.plot(min_values)
    plt.show()'''

  book = openpyxl.load_workbook(file_path)
  del book['Sheet']
  book.save(file_path)
```

附录 5：问题三的遗传算法的 Python 代码

```
# 问题3非线性0-1整数规划的遗传算法求解

import numpy as npy
from scipy.optimize import minimize
from deap import base, creator, tools, algorithms
import pandas as pd
import matplotlib.pyplot as plt
import time

time1 = time.time()
# 定义问题中的常数
e = [0.1 for _ in range(8)] # 8个零件的次品率
p = [2, 8, 12, 2, 8, 12, 8, 12] # 8个零件的采购价
tc = [1, 1, 2, 1, 1, 2, 1, 2] # 8个零件的检测价
eps = [0.1 for _ in range(3)] # 半成品装配的次品率
ep = 0.1                    # 成品装配的次品率
assps = [8, 8, 8]           # 半成品的加工费
assp = 8                    # 成品的加工费
tcps = [4, 4, 4]            # 半成品的检测价
tcp = 6                     # 成品的检测价
dassps = [6, 6, 6]          # 半成品的拆解费
dassp = 10                  # 成品的拆解费
```

```
re = 40                    # 成品调换损失

# 创建遗传算法的适应度函数，目标是最小化问题
creator.create("FitnessMin", base.Fitness, weights=(-1.0,))
creator.create("Individual", list, fitness=creator.FitnessMin)

# 初始化种群的个体，整数变量(0或1)
def init_individual():
  # 整数变量部分：19个决策变量
  int_vars = npy.random.randint(0, 2, 19).tolist()
  return creator.Individual(int_vars)

# 定义目标函数(成本)的计算
def objective(solution, t1, t2, t3, t4, t5, t6, t7, t8,
     t1_, t2_, t3_, t4_, t5_, t6_, t7_, t8_,
     t1__, t2__, t3__, t4__, t5__, t6__, t7__, t8__,
     tp1, tp2, tp3, tp,
     tp1_, tp2_, tp3_,
     dp1, dp2, dp3, dp,
     dp1_, dp2_, dp3_
     ):
  t = [t1, t2, t3, t4, t5, t6, t7, t8]
  t_ = [t1_, t2_, t3_, t4_, t5_, t6_, t7_, t8_]
  t__ = [t1__, t2__, t3__, t4__, t5__, t6__, t7__, t8__]
  tps = [tp1, tp2, tp3]
  tps_ = [tp1_, tp2_, tp3_]
  dps = [dp1, dp2, dp3]
  dps_ = [dp1_, dp2_, dp3_]

  n1, n2, n3, n4, n5, n6, n7, n8, \
  l1, l2, l3, l4, l5, l6, l7, l8, \
  l1_, l2_, l3_, l4_, l5_, l6_, l7_, l8_, \
  np1, np2, np3, np, \
  lp1, lp2, lp3, \
  e1h, e2h, e3h, e4h, e5h, e6h, e7h, e8h, \
  e1hh, e2hh, e3hh, e4hh, e5hh, e6hh, e7hh, e8hh, \
  e1_, e2_, e3_, e4_, e5_, e6_, e7_, e8_, \
  e1__, e2__, e3__, e4__, e5__, e6__, e7__, e8__, \
  ep1h, ep2h, ep3h, eph, \
  ep1hh, ep2hh, ep3hh, \
  ep1_, ep2_, ep3_ = solution
```

```
n = [n1, n2, n3, n4, n5, n6, n7, n8]
l = [l1, l2, l3, l4, l5, l6, l7, l8]
nps = [np1, np2, np3]
lps = [lp1, lp2, lp3]
eh = [e1h, e2h, e3h, e4h, e5h, e6h, e7h, e8h]
ehh = [e1hh, e2hh, e3hh, e4hh, e5hh, e6hh, e7hh, e8hh]
e_ = [e1_, e2_, e3_, e4_, e5_, e6_, e7_, e8_]
ephs = [ep1h, ep2h, ep3h]
eps_ = [ep1_, ep2_, ep3_]

obj = 0

for i in range(8):
  obj += n[i] * p[i]          # 采购费用
  obj += n[i] * tc[i] * t[i] # 采购的零件是否检测产生的费用
for i in range(3):
  obj += nps[i] * tps[i] * tcps[i] # 半成品是否检测产生的费用
  obj += nps[i] * assps[i]   # 半成品的装配费用

  # 半成品是否拆解产生的费用
  obj += (nps[i] * tps[i] * dps[i] * ephs[i] + np * eph * dp *
      eps_[i] * tps_[i] * dps_[i]) * dassps[i]

obj += np * tp * tcp # 成品是否检测产生的费用

# 拆解后回流的零件是否检测产生的费用
obj += np1 * tp1 * dp1 * ep1h * (t1_*tc[0] + t2_*tc[1] + t3_*tc
    [2])
obj += np2 * tp2 * dp2 * ep2h * (t4_*tc[3] + t5_*tc[4] + t6_*tc
    [5])
obj += np3 * tp3 * dp3 * ep3h * (t7_*tc[6] + t8_*tc[7])

obj += np * eph * dp * ep1_ * tp1_ * dp1_ * (t1__ * tc[0] + t2__
    * tc[1] + t3__ * tc[2])
obj += np * eph * dp * ep2_ * tp2_ * dp2_ * (t4__ * tc[3] + t5__
    * tc[4] + t6__ * tc[5])
obj += np * eph * dp * ep3_ * tp3_ * dp3_ * (t7__ * tc[6] + t8__
    * tc[7])

# 拆解后回流的半成品是否检测产生的费用
```

```
    obj += np * dp * eph * (tp1_*tcps[0] + tp2_*tcps[1] + tp3_*tcps
        [2])

    # 成品的装配费用与是否拆解成品产生的费用
    obj += np * assp
    obj += np * dp * eph * dassp

    # 是否产生调换损失
    obj += np * (1-tp) * eph * re

    return obj

# 定义方程
def mse(vars, t1, t2, t3, t4, t5, t6, t7, t8,
        t1_, t2_, t3_, t4_, t5_, t6_, t7_, t8_,
        t1__, t2__, t3__, t4__, t5__, t6__, t7__, t8__,
        tp1, tp2, tp3, tp,
        tp1_, tp2_, tp3_,
        dp1, dp2, dp3, dp,
        dp1_, dp2_, dp3_
        ):

    # 决策变量
    t = [t1, t2, t3, t4, t5, t6, t7, t8]
    t_ = [t1_, t2_, t3_, t4_, t5_, t6_, t7_, t8_]
    t__ = [t1__, t2__, t3__, t4__, t5__, t6__, t7__, t8__]
    tps = [tp1, tp2, tp3]
    tps_ = [tp1_, tp2_, tp3_]
    dps = [dp1, dp2, dp3]
    dps_ = [dp1_, dp2_, dp3_]

    # 非决策变量
    n1, n2, n3, n4, n5, n6, n7, n8, \
    l1, l2, l3, l4, l5, l6, l7, l8, \
    l1_, l2_, l3_, l4_, l5_, l6_, l7_, l8_, \
    np1, np2, np3, np, \
    lp1, lp2, lp3, \
    e1h, e2h, e3h, e4h, e5h, e6h, e7h, e8h, \
    e1hh, e2hh, e3hh, e4hh, e5hh, e6hh, e7hh, e8hh, \
    e1_, e2_, e3_, e4_, e5_, e6_, e7_, e8_, \
    e1__, e2__, e3__, e4__, e5__, e6__, e7__, e8__, \
```

```
ep1h, ep2h, ep3h, eph, \
ep1hh, ep2hh, ep3hh, \
ep1_, ep2_, ep3_ = vars

n = [n1, n2, n3, n4, n5, n6, n7, n8]
l = [l1, l2, l3, l4, l5, l6, l7, l8]
l_ = [l1_, l2_, l3_, l4_, l5_, l6_, l7_, l8_]
nps = [np1, np2, np3]
lps = [lp1, lp2, lp3]
eh = [e1h, e2h, e3h, e4h, e5h, e6h, e7h, e8h]
ehh = [e1hh, e2hh, e3hh, e4hh, e5hh, e6hh, e7hh, e8hh]
e_ = [e1_, e2_, e3_, e4_, e5_, e6_, e7_, e8_]
e__ = [e1__, e2__, e3__, e4__, e5__, e6__, e7__, e8__]
ephs = [ep1h, ep2h, ep3h]
ephhs = [ep1hh, ep2hh, ep3hh]
eps_ = [ep1_, ep2_, ep3_]

equations = []

# 总流量方程
for i in range(3):
  for j in range(3):
    k = i * 3 + j
    if not k == 8:
      equations.append(n[k] * (1 - e[k] * t[k]) + l[k] + l_[k] -
          nps[i])
      equations.append(nps[i] * tps[i] * dps[i] * ephs[i] * (1 - e_
          [k] * t_[k]) - l[k])
      equations.append(l_[k] - dps_[i] * (np * eph - lps[i]) * (1 -
           t__[k] * e__[k]))
  equations.append(nps[i] * (1 - ephs[i] * tps[i]) + lps[i] - np)
  equations.append(np * dp * eph * (1 - eps_[i] * tps_[i]) - lps[i
      ])
equations.append(np * (1 - eph) - 1)

# 次品率方程
for i in range(3):
  for j in range(3):
    k = i * 3 + j
    if not k==8:
```

```
            equations.append(nps[i] * eh[k] - n[k] * e[k] * (1 - t[k]) -
                l[k] * e_[k] * (1 - t_[k]) - l_[k] * e__[k] * (1 - t__[k])
                )
            equations.append(eh[k] * tps[i] * dps[i] - tps[i] * dps[i] *
                ephs[i] * e_[k])
            equations.append(eh[k] * nps[i] * (1 - tps[i]) + ehh[k] * np
                * eph * dp * (1 - tps_[i]) - np * ehh[k])
            equations.append(ehh[k] * dp * tps_[i] * dps_[i] - eph * dp *
                 eps_[i] * tps_[i] * dps_[i] * e__[k])
        equations.append(np * ephhs[i] - nps[i] * ephs[i] * (1 - tps[i])
             - lps[i] * eps_[i] * (1 - tps_[i]))
        equations.append(ephhs[i] * dp - eph * eps_[i] * dp)
    equations.append(ep1h - eps[0] * (1 - e1h) * (1 - e2h) * (1 - e3h
        ) - (e1h + e2h + e3h - e1h * e2h - e1h * e3h - e2h * e3h + e1h
         * e2h * e3h))
    equations.append(ep2h - eps[1] * (1 - e4h) * (1 - e5h) * (1 - e6h
        ) - (e4h + e5h + e6h - e4h * e5h - e4h * e6h - e5h * e6h + e4h
         * e5h * e6h))
    equations.append(ep3h - eps[2] * (1 - e7h) * (1 - e8h) - (e7h +
        e8h - e7h * e8h))
    equations.append(eph - ep * (1 - ep1hh) * (1 - ep2hh) * (1 -
        ep3hh) - (ep1hh + ep2hh + ep3hh - ep1hh * ep2hh - ep1hh *
        ep3hh - ep2hh * ep3hh + ep1hh * ep2hh * ep3hh))

    return sum(x**2 for x in equations)/73

# 定义目标函数的评估
def evaluate(individual):

    # 从遗传算法的输出中解码决策变量
    t = individual[:8]
    ones1 = npy.ones_like(t)
    t_ = ones1 - t # 考虑决策变量的约束
    t__ = ones1 - t
    tps = individual[8:11]
    ones2 = npy.ones_like(tps)
    tp = individual[11]
    tps_ = ones2 - tps # 考虑决策变量的约束
    dps = individual[12:15]
    dp = individual[15]
    dps_ = individual[16:19]
```

```
    initial_guess = [1 for _ in range(8)] + [0.5 for _ in range(65)]

    # 通过最小化mse解得非决策变量方程的解，即非决策变量的值
    solution = minimize(lambda vars: mse(vars, *t, *t_, *t__, *tps,
        tp, *tps_, *dps, dp, *dps_),
            initial_guess,
            options={'maxiter': 10000, 'maxfun': 30000},
            tol=1e-6,

            # 方程求解的限制，采购流量为>1的连续值，其余流量>0，而次品率为(0，1)
                间的连续值
            bounds=[(1, None)for i in range(8)] + [(0, None) for _ in
                range(23)] + [(0, 1) for _ in range(42)],
            )

    if solution.success and mse(solution.x, *t, *t_, *t__, *tps, tp,
        *tps_, *dps, dp, *dps_) < 1e-5: # 保证方程求解是准确的
      vars_solution = solution.x # 从优化结果中提取解
      print(f't: {t}, t_: {t_}, t__: {t__}, tps: {tps}, tp:{tp}, tps_:
          {tps_}, dps: {dps}, dp:{dp}, dps_:{dps_}')
      print(f'n1: {vars_solution[0]}, n2: {vars_solution[1]}, n3: {
          vars_solution[2]}, n4: {vars_solution[3]}, n5: {vars_solution
          [4]}, n6: {vars_solution[5]}, n7: {vars_solution[6]}, n8: {
          vars_solution[7]}')
      print(f'MSE: {mse(vars_solution, *t, *t_,*t__, *tps, tp, *tps_,
          *dps, dp, *dps_)}')

      # 计算得到目标函数
      obj = objective(vars_solution, *t, *t_,*t__, *tps, tp, *tps_, *
          dps, dp, *dps_)
      print('obj:', obj)
      return obj,

    else:
      return float('inf'), # 如果优化失败，返回无穷大

# 注册遗传算法的主要操作
toolbox = base.Toolbox()
toolbox.register("individual", init_individual)
```

```
toolbox.register("population", tools.initRepeat, list, toolbox.
    individual)
toolbox.register("evaluate", evaluate)
toolbox.register("mate", tools.cxUniform, indpb=0.5) # 使用均匀交叉

# 为整数变量定义变异操作，确保变异后值仍为0或1
def mutate_integer(individual, indpb):
  for i in range(19):
    if npy.random.random() < indpb:
      individual[i] = 1 - individual[i] # 0变1，1变0
  return individual,

# 注册整数的变异
toolbox.register("mutate", mutate_integer, indpb=0.2)

# 注册选择操作（锦标赛选择）
toolbox.register("select", tools.selTournament, tournsize=3)

# 定义遗传算法的执行步骤
def run_ga():
  # 设置种群数为100
  population = toolbox.population(n=100)
  # 初始化一个列表来存储每一代的最低值
  min_values_per_generation = []

  # 进行40代进化
  for gen in range(40):
    offspring = algorithms.varAnd(population, toolbox, cxpb=0.7,
        mutpb=0.2)
    fits = toolbox.map(toolbox.evaluate, offspring)
    for ind, fit in zip(offspring, fits):
      ind.fitness.values = fit
    population = toolbox.select(offspring, k=len(population))
    now_best = tools.selBest(population, k=1)[0]
    # 记录这一代的最低值
    min_value = evaluate(now_best)
    min_values_per_generation.append(min_value)
    print(f"Generation {gen}: Min value = {min_value}")

  return population, min_values_per_generation
```

```python
# 执行遗传算法
final_population, min_values = run_ga()
time2 = time.time()

print('running time:', time2 - time1)

# 输出最优个体和其目标值
best_individual = tools.selBest(final_population, k=1)[0]
print("Best individual:", best_individual)
print("Objective value:", evaluate(best_individual))

# 将结果保存至excel中
t = best_individual[:8]
ones1 = npy.ones_like(t)
t_ = ones1 - t
t__ = ones1 - t
tps = best_individual[8:11]
ones2 = npy.ones_like(tps)
tp = best_individual[11]
tps_ = ones2 - tps
dps = best_individual[12:15]
dp = best_individual[15]
dps_ = best_individual[16:19]

result_1 = pd.DataFrame({'t': t, 't\'': t_, 't\'\'': t__}, index=
    range(1, 9))
result_2 = pd.DataFrame({'tps': tps, 'tps\'': tps_, 'dps': dps, '
    dps\'': dps_}, index=range(1, 4))
result_3 = pd.DataFrame({'tp': tp, 'dp': dp}, index=[0])
result = pd.concat([result_1, result_2, result_3])
result.to_excel('./result/problem3.xlsx')


# 绘制优化曲线
plt.plot(min_values)
plt.title('Every generation’ s optimal individual (decision)
    objective function (cost).')
plt.xlabel('generational number')
plt.ylabel('cost')
plt.show()
```

附录 6：问题四中计算置信区间的 Python 代码

```python
import scipy.stats as stats
def find_probability(n, threshold):
  # 设定概率 p 的初始范围
  p_min, p_max = 0.1, 1
  precision = 1e-6 # 解的精度
  m = int(n * 0.05) # 次品数量

  while p_max - p_min > precision:
    p_mid = (p_min + p_max) / 2
    cumulative_prob = sum(stats.binom.pmf(i, n, p_mid) for i in
        range(m)) # i 从 0 到 m
    if cumulative_prob >= threshold:
      p_min = p_mid
    else:
      p_max = p_mid

  right_bound = p_mid
  p_min, p_max = 0, 0.1
  while p_max - p_min > precision:
    p_mid = (p_min + p_max) / 2
    cumulative_prob = sum(stats.binom.pmf(i, n, p_mid) for i in
        range(m, n+1)) # i 从 m 到 n
    if cumulative_prob >= threshold:
      p_max = p_mid
    else:
      p_min = p_mid

  left_bound = p_mid

  return left_bound, right_bound

n = 100 # 抽样数
threshold = 0.05 # 概率阈值

left, right = find_probability(n, threshold)

print(f"95% CI for n={n} is [{left}, {right}]")
```